

HostelHub — Master Frontend Build Prompt

For: GitHub Copilot Agent Mode (VS Code) — Flutter

***How to use this file:** Save it as `.github/copilot-instructions.md` in the repo root (Copilot loads it automatically for every request), or paste it directly into Copilot Chat in Agent Mode at the start of a session. It supersedes nothing in your original design doc — it consolidates it with the 6 reference screenshots (Schedule, Hamburger Menu, Alerts, Profile, My Account) into one buildable spec. If the prompt is too large for one pass, use Section 16 to run it in phases.*

0. Assumptions Made While Writing This Prompt

State these back to the project owner if any are wrong — do not silently guess further.

- Framework: Flutter (Dart).** Inferred from the original doc's own code examples (`Color(...)` , `EdgeInsets.all(...)` , `BorderRadius.circular(...)` , `AppColors.primary`). If the actual repo is a different framework, keep every token/rule below identical and translate only the syntax.
- State management is unspecified** by the source doc. Inspect the existing project first. If a solution is already in use (Provider, Riverpod, Bloc, GetX), continue with it. If starting clean, default to **Provider** — simple, common in Flutter, easy to replace later. This is a suggestion, not a rule to defend.
- Two distinct overlays exist**, not one. The reference screenshots show:
 - A **Hamburger Drawer** (slides from the **left**, opened by `☰`, navigation-first — profile summary + menu list + logout).
 - An **Account Panel** (slides from the **right**, opened by tapping the **avatar photo** in the Home header, settings-first — profile edit, password, email, language, theme).

The original doc's structure tree only lists "Hamburger Menu." Build both as separate reusable components; see Section 8.3.

- Screens with no explicit spec in the source doc** (Room, Mess, Laundry, Repair, Events, Resources, Settings, Notification Preferences, Help & Support, Report a Problem, About, My Hostel Details) have been designed below **by extending the existing system** — reusing the same cards, status trackers, and list-row patterns already defined for Home/Schedule/Alerts/Profile. Per the source doc's own Section 1, flag any of these to the project owner before treating them as final; they are the one part of this prompt that isn't sourced from an approved reference.
- Header navigation convention:** `☰` (hamburger) appears only on the 4 root tab screens — Home, Schedule, Alerts, Profile. Every drill-in screen (a Hostel Service page, Settings, About, a detail view opened from a card) uses a back arrow (`←`) instead. This keeps "exactly 4 items in bottom nav" intact while making deep screens navigable.
- "Smart Alert" accent color:** the Alerts reference screenshot renders the Smart Alert category tag/dot in a tone that reads slightly more indigo/purple than the documented Primary Blue (`#326DF0`). The source doc defines no separate token for this. Default to **Primary Blue** for Smart Alerts

until the project owner confirms whether a distinct "smart" accent is intentional — do not invent a new hex value unilaterally (this mirrors the source doc's own color-approval rule in Section 5/56).

1. Your Role (Non-Negotiable)

You are an AI coding agent working alongside three human developers on **one shared Flutter codebase** for HostelHub — a hostel life utility manager for students. The app must look like it was built by one professional team, not three people improvising separately.

Before writing, modifying, or generating any frontend code:

1. Read this entire file.
2. Inspect the existing project structure, theme, and shared widgets.
3. Reuse existing components before creating new ones.
4. Never invent a new visual style, introduce a new color/font, duplicate a shared component, or redesign a screen someone else already built.
5. Never modify another team member's feature unless integration requires it.
6. Never touch this instructions file — don't weaken, shorten, or "helpfully" rewrite its rules — unless the project owner explicitly asks you to.

Most important rule: do not alter the existing app's size, colors, interface, or UI design — screen dimensions, layout, spacing, typography, colors, borders, radius, shadows, icons, navigation, card design, button design, interaction patterns, or animations — unless the project owner explicitly requests that specific change. You are **extending** HostelHub, not redesigning it. This applies even if you believe a different choice would be more modern, cleaner, or more "professional" — your design opinions do not override the owner's system. If you believe a change is genuinely required for functionality, accessibility, or technical compatibility, **report it and wait** rather than silently reworking the UI.

There is **no payments/fees/wallet/billing feature** anywhere in this app. Never add one, even as a placeholder.

2. Tech Stack & Architecture

Stack: Flutter (Dart), mobile-first (iOS + Android), responsive to varying screen sizes — never hard-coded to one screenshot's resolution.

Layering:

```
UI (screens/widgets)
  ↓
Model
  ↓
Repository / Service layer
  ↓
Mock data (now) → API/backend (later)
```

The UI must not need a rewrite when a real backend is connected later — talk to repositories/services, never to mock data directly from widgets.

Suggested folder structure (adapt to whatever already exists in the repo — don't restructure a working project to match this exactly):

```
lib/
├─ main.dart
├─ app.dart
├─ theme/
│   ├─ app_colors.dart
│   ├─ app_typography.dart
│   ├─ app_spacing.dart
│   ├─ app_radius.dart
│   ├─ app_shadows.dart
│   └─ app_theme.dart
├─ models/
├─ repositories/           # or services/ - match existing naming if present
├─ mock_data/
├─ widgets/               # shared, reusable across features
├─ navigation/            # routes, bottom nav, drawer/panel controllers
└─ screens/
    ├─ entry/              # splash, onboarding, login
    ├─ home/
    ├─ schedule/
    ├─ alerts/
    ├─ profile/
    ├─ services/           # room, mess, laundry, repair, events, resources
    └─ secondary/          # settings, notification_prefs, help, report, about, ho
stel_details
```

3. Design Tokens

3.1 Colors

```
class AppColors {  
    // Primary  
    static const primary      = Color(0xFF326DF0);  
    static const primaryDark  = Color(0xFF1D43B6);  
    static const primaryLight = Color(0xFF94B9F2);  
    static const deepNavy     = Color(0xFF061258);  
  
    // Base  
    static const background    = Color(0xFFFFAFCFF);  
    static const surface       = Color(0xFFFFFFFF);  
    static const surfaceSoft   = Color(0xFFFF4F8FF);  
    static const border        = Color(0xFFDDE7F5);  
    static const divider       = Color(0xFFE6EDF7);  
  
    // Text  
    static const textPrimary   = Color(0xFF061258);  
    static const textSecondary = Color(0xFF5B6B85);  
    static const textTertiary  = Color(0xFF8A98AD);  
    static const textDisabled  = Color(0xFFB6C0CF);  
  
    // Semantic  
    static const success       = Color(0xFF26A66A);  
    static const successLight  = Color(0xFFEAF8F1);  
    static const warning       = Color(0xFFFF2A51A);  
    static const warningLight  = Color(0xFFFFF5DE);  
    static const error         = Color(0xFFE34D59);  
    static const errorLight    = Color(0xFFFFF0F1);  
    static const info          = Color(0xFF326DF0);  
    static const infoLight     = Color(0xFFEEF5FF);  
  
    // Service colors  
    static const roomIcon      = Color(0xFF1D43B6);  
    static const roomBg        = Color(0xFFEEF4FF);  
    static const messIcon      = Color(0xFF326DF0);  
    static const messBg        = Color(0xFFFF0F6FF);  
    static const laundryIcon   = Color(0xFF061258);  
    static const laundryBg     = Color(0xFFEAF1FF);  
    static const repairIcon    = Color(0xFFE04B55);  
}
```

```

static const repairBg      = Color(0xFFFFF0F1);
static const eventsIcon    = Color(0xFF35A86F);
static const eventsBg      = Color(0xFFEDF9F2);
static const resourcesIcon = Color(0xFF22A6A6);
static const resourcesBg   = Color(0xFFEDFAFA);
}

```

Usage rules: Blue is the **only** brand/action color — active nav, primary buttons, selected tabs/filters, links, active states. Never use blue as a generic "pretty" fill on every card. Semantic colors stay soft (use the *Light variants for backgrounds, full-strength only for icons/text/dots). Never introduce a color outside this palette without owner approval.

3.2 Typography

Font family: **Inter** (fallback: system sans-serif). No decorative fonts, ever.

Style	Size	Weight	Used for
Display / Hero	28px	700	Splash, rare hero moments
Page Title	24px	700	Header titles, greeting
Section Heading	20px	700	Section titles ("For You", "Quick Access")
Card Heading	16px	600	Card/list-item titles
Body	15px	400	Body copy
Body Medium	15px	500	Emphasized body copy
Small / Supporting	13px	400	Secondary/supporting lines
Caption	12px	500	Timestamps, captions
Navigation Label	13px	500	Bottom nav labels

Secondary info gets the secondary text color, not a smaller font size. Don't use more font weights than this table defines.

3.3 Spacing (4pt system)

Token	Value
XS	4px
SM	8px
MD	12px
LG	16px
XL	20px
XXL	24px
XXXL	32px

Page horizontal padding: 16–20px. No off-grid values (13, 17, 23, 27px...) without a real layout reason.

3.4 Border Radius

Element	Radius
Small components	8px
Buttons	12px
Standard cards	16px
Large cards	18–20px
Bottom sheets / drawers	24px

Rounded throughout; avoid sharp rectangular cards.

3.5 Cards & Shadows

Default card: `surface` background, 1px `border` stroke, 16px radius, 16px padding, **extremely subtle** shadow (barely-there elevation — never a floating 3D look).

```
class AppSpacing {
  static const xs = 4.0, sm = 8.0, md = 12.0, lg = 16.0,
               xl = 20.0, xxl = 24.0, xxxl = 32.0;
}

class AppRadius {
  static const sm = 8.0, button = 12.0, card = 16.0,
               cardLg = 20.0, sheet = 24.0;
}
```

3.6 Buttons

Variant	Background	Text	Notes
Primary	<code>primary</code>	White	Height ~48px, radius 12px
Secondary	<code>primaryLight</code>	<code>primary</code>	Same height/radius
Outlined	Transparent	<code>primary</code>	1px <code>primary</code> border

Same button styles everywhere — never a one-off variant for a single page.

3.7 Inputs

Height 48–52px, radius 12px, `border` stroke, `surface` background. Focused → `primary` border.
Error → `error` border + `error` supporting text.

3.8 Iconography

One consistent icon family app-wide — clean, rounded/outlined, consistent stroke weight. Never mix icon libraries. Emoji (👋, 🌞) are **content** (greetings, weather) — not the UI icon system.

4. Core Reusable Components (build these before any screen)

Component	Purpose
<code>AppHeader</code>	☰ + title (+ optional subtitle) + up to one right-side action slot (icon or icon pair) — see Section 8 for per-screen config
<code>AppBackHeader</code>	← + title, for drill-in screens
<code>AppBottomNavigation</code>	4 fixed items: Home, Schedule, Alerts, Profile; active = filled icon + <code>primary</code> ; inactive = outline icon + gray; supports a badge on Alerts
<code>NotificationBadge</code>	Small red (<code>error</code>) circle with white bold count, anchored top-right of an icon
<code>AppCard</code>	Base card per Section 3.5
<code>ServiceCard</code>	Icon (in its service-colored circle/square) + name + short description + <code>></code> chevron
<code>SectionHeader</code>	Title (Section Heading style) + optional "View All" link in <code>primary</code>
<code>PrimaryButton</code> / <code>SecondaryButton</code> / <code>OutlinedButton</code>	Per Section 3.6
<code>AppInputField</code>	Per Section 3.7
<code>StatusBadge</code>	Small pill, colored per status/category (see per-screen notes — status pills inherit the service color , not one fixed "upcoming" color)
<code>ScheduleCard</code>	Icon box + title + category • location + time range + <code>StatusBadge</code>
<code>AlertCard</code>	Icon box + eyebrow category label (colored) + title + description (with inline emphasis on key details) + timestamp + unread dot
<code>HighlightCard</code>	Tinted card: date/time + icon + title + location + "View Details >"
<code>TimelineIndicator</code>	Vertical connector line + colored dot per timeline row
<code>EmptyState</code>	Icon/illustration + friendly message, same typography/card system as everything else
<code>LoadingState</code>	Skeleton/shimmer placeholders — never a full-screen blocking spinner unless unavoidable
<code>ErrorState</code>	"Something went wrong" message + <code>PrimaryButton</code> labeled "Try Again"
<code>ProfileHeader</code>	Avatar (+ optional camera-overlay edit affordance) + name + ID + meta lines
<code>HamburgerDrawer</code>	Left-slide navigation drawer — see 8.3
<code>AccountPanel</code>	Right-slide account/settings panel — see 8.3
<code>InfoRow</code>	Icon + label + value (+ optional chevron) — used across Profile, Account Panel, My Hostel Details

Before creating any new component, search the project for one of these. Do not build `LaundryCard`, `MessCard`, `RepairCard`, `EventsCard` etc. as separate visual systems — configure `ServiceCard` / `ScheduleCard` / `AlertCard` via properties instead.

5. Navigation Map

Splash

- └ (first launch) → Onboarding → Login → Main App
- └ (returning, logged in) → Main App
- └ (returning, logged out) → Login → Main App

Main App = Bottom Tab Navigator (Home | Schedule | Alerts | Profile)

Each tab root can push drill-in screens (back-arrow header):

- Home → Room / Mess / Laundry / Repair / Events / Resources (via Quick Access)
 - individual "For You" / notice / event detail pushes
- Schedule → View Calendar, event/highlight detail
- Alerts → alert detail (if needed)
- Profile → My Hostel Details, Settings, Notification Preferences, Help & Support, Report a Problem, About

Global overlays (reachable from any of the 4 tab roots):

- ☰ → HamburgerDrawer (slides from left)
- 🔔 → Alerts tab (bell in header and bottom-nav Alerts both lead to the same Alerts experience)
- 👤 (avatar, Home header only) → AccountPanel (slides from right)

6. Screen Specifications

6.1 ENTRY

Splash

Minimal. Centered brand lockup: **HostelHub** wordmark ("Hostel" in `textPrimary`, "Hub" in `primary`) + tagline "Your hostel, organized." on `background`. Primary brand colors only. No extra content, no loading chrome beyond what's needed. Auto-advances after a short delay.

Onboarding

Three benefit slides, same design system (cards/typography/spacing), swipeable, with pagination dots + Skip / Next / Get Started actions using `PrimaryButton` / `OutlinedButton`:

1. **Everything in one place**
2. **Stay updated**
3. **Know what's next**

Login

Clean form, vertically centered: Email/Student ID field, Password field (`AppInputField`), `PrimaryButton` "Login". Keep it minimal — no unrequested fields.

6.2 MAIN APP

Home

Header uses the special Home variant: `≡ HostelHub / Your hostel, organized.` on the left, **both** a  (with `NotificationBadge` when unread alerts exist) and  avatar (opens `AccountPanel`) on the right.

Vertically scrollable, in this exact order:

1. **Greeting** — "Good {Morning/Afternoon/Evening}, {Name}  " at Page Title size (24/700), current date below at Body size (15/400) — reference shows full format ("Friday, 14 August 2025"); greeting must always read visually larger/heavier than the date.
2. **Weather Card** — compact, non-dominant card (e.g. "  29°C · Chennai"), toward the top area. Mock data is fine for the prototype.
3. **Room Snapshot** — per the reference screenshots this renders as a **solid filled card** (deep navy/primary-dark background, white text) rather than merely "soft-tinted" as the written spec states — follow the image. Content: "Block B · Room 304", "3rd Floor · 2 Roommates", "View Room >" pill action. Tapping navigates to the Room service page.
4. **For You** — `SectionHeader` ("For You" / "View All"). Personalized/predictive cards, each a full-width row (icon box + title + subtitle + chevron), colored per originating service (e.g. Laundry Slot, Repair Scheduled, Mess Update). This is the "smart alerts" surface — see 6.4.
5. **Quick Access** — `SectionHeader` ("Quick Access" / "View All"). 6 `ServiceCard` tiles in a 3-column, 2-row grid: Room, Mess, Laundry / Repair, Events, Resources.
6. **Today's Schedule** — `SectionHeader` ("Today's Schedule" / "View Schedule"). Compact timeline preview (time + icon + label rows), lighter-weight than the full Schedule page's cards.
7. **Today's Menu** — Breakfast / Lunch / Snacks / Dinner, each with icon + name + menu text + optional timing, Mess service color.
8. **Active Repair** — current repair issue with a compact status tracker: `Reported ✓ → Assigned ✓ → Scheduled ● → Completed ○`. Statuses must be distinguishable without relying on color alone (icon/label pairing, not color-only).

9. **Important Notice** — soft red/pink background (never bright red), title "Important Notice", message, "View All >".
10. **Upcoming Event** — Events green language, title + date/time/location, "View Events >".
11. **Bottom Navigation.**

Never let Payments/Fees/Billing appear anywhere on this page.

Schedule

Matches the reference screenshots closely:

- Header:  **Schedule** with a calendar icon on the right (no subtitle on this page).
- Tab row: **Today** (selected — solid **primary** pill, white text, fully rounded) / Tomorrow / This Week (unselected — white background, **textPrimary**), all inside a soft outer container.
- Filter chip row: All (grid icon) / Mess (fork-knife) / Laundry (basket) / Repair (wrench) / Events (calendar) — white rounded-full pills, colored icon per service, selected state = **primary** border + **primary** text.
- Date row: calendar icon + "Friday, 14 August 2025" (bold, left) and "View Calendar >" (**primary**, right).
- **Timeline:** vertical connector line; each node is a colored dot (color = that event's service color) with the time to the left (e.g. "08:00 / AM") and a card to the right containing: icon box (service-colored), bold title, "Category • Location" line, time range, and a **StatusBadge** pill on the far right — **the badge's color matches the event's service color**, it is not a single universal "upcoming = orange" rule (Breakfast/Lunch/Dinner badges are orange/Mess-colored, Laundry Slot's badge is blue, Badminton's is green/Events-colored, Fan Repair's is red/Repair-colored).
- **Upcoming Highlights** (this section is present in the reference screenshots but was not written into the original text spec — build it as part of the canonical Schedule page): a bordered outer card containing a header row (bookmark icon + "Upcoming Highlights" + "View All" in **primary**) and two side-by-side tinted **HighlightCard**s (e.g. a primary-tinted card and a green/Events-tinted card), each showing date/time, icon, event title, location, and "View Details >" in that card's accent color.
- Bottom Navigation, Schedule tab active.

Alerts

Reachable from both the bottom nav Alerts tab and the header bell icon (same destination).

- Header:  **Alerts** / "Stay updated with important things", with  search and  filter icons on the right.
- Filter pill row: All (selected, solid **primary** pill) / Smart Alerts (sparkle icon) / Notices (megaphone icon) / Updates (bell icon).
- Row: "Unread" label (left) / "Mark all as read" with a check-circle icon (right, **primary**).
- **Unread alert cards** (**AlertCard**): icon box (colored per category) · eyebrow category label (small caps, colored — e.g. "SMART ALERT", "MESS UPDATE", "NOTICE") with the time top-right · bold title · 1–2 line description with key details emphasized inline (e.g. a time or date rendered bold/colored within the sentence) · a small calendar icon + date (+ location where relevant) on the bottom row · a colored dot on the far right marking it unread (dot color = category color).

- **Earlier** section header, visually quieter than "Unread". Same `AlertCard` structure, but the dot is neutral gray (read) and the top-right shows a relative date ("13 Aug") instead of a time. Categories seen in reference: Event Update (green), Hostel Notice (red), Security Update (use `primary` /indigo pending owner confirmation per Assumption 6).
- Bottom Navigation, Alerts tab active with `NotificationBadge` when unread count > 0.

Profile

One of the 4 root tabs.

- Header: `≡ Profile` / "Manage your account and hostel details",  with `NotificationBadge` on the right.
- **Profile card** — soft `primaryLight`-tinted background: avatar (with a small camera-icon edit affordance), name (bold), student ID (`primary`), course, batch (e.g. "2024 – 2028 Batch"), and an outlined "Edit Profile" pill button.
- **Personal Information** (`SectionHeader` + icon): `InfoRow` list — Full Name, Email Address, Mobile Number, Date of Birth, Blood Group — each icon + label + value + chevron.
- **Hostel Information** (`SectionHeader` + icon): a 4-column icon grid (Hostel/Block B, Room Number/304, Floor/3rd Floor, Room Type/Triple Sharing) plus a "View Hostel Details" row (icon + title + subtitle "Hostel rules, timings, wardens and more" + chevron) linking to the My Hostel Details screen (6.5).
- **Roommates** (`SectionHeader` + icon): list rows — avatar + name (bold) + student ID (`primary`) + two circular action buttons (call, message).
- **Quick Actions** (`SectionHeader`): Privacy & Security, Notification Preferences, Export My Data — each an `InfoRow` with icon + title + subtitle + chevron.
- Bottom Navigation, Profile tab active.

6.3 GLOBAL OVERLAYS

Hamburger Drawer

Follows the reference screenshot closely. Slides in from the **left**, ~80–90% of screen width, white background, rounded leading (right-side) corners, dimmed overlay over the rest of the screen, clear × close affordance.

Content, top to bottom:

1. Brand lockup (icon + "HostelHub" + "Your hostel, organized.") with × close icon.
2. Profile summary row: avatar, name, student ID (`primary`), "Block B • Room 304", floor.
3. **ACCOUNT** — My Profile, My Hostel Details
4. **ACTIVITY** — Notifications (with `NotificationBadge`), All Notices, My Schedule
5. **PREFERENCES** — Settings, Notification Preferences
6. **SUPPORT** — Help & Support, Report a Problem, About HostelHub
7. **Logout** — red icon + red bold label, on a soft red-tinted row, visually separated from the menu list above.

8. Version string centered at the very bottom (e.g. "Version 1.0.0").

Each menu row: icon in a soft `primaryLight` / `infoLight` circle, label, trailing chevron. Generous vertical spacing, subtle section dividers, uppercase small-caps section labels in `textTertiary`.

No payment-related entries in this menu.

Account Panel

New component (see Assumption 3) — slides in from the **right**, opened by tapping the avatar in the Home header. Distinct from the Hamburger Drawer: this panel is about *account settings*, not navigation. Content, top to bottom:

1. Header: "My Account" / "Manage your profile and account", ✕ close icon.
2. Profile card: avatar (camera-edit affordance), name, student ID, course, batch.
3. 3-column row: Block ("Hostel"), Room Number ("Your Room"), Floor.
4. **ACCOUNT** — Edit Profile, Change Password, Linked Email (value shown), Mobile Number (value shown), Language (value shown, e.g. "English").
5. **PREFERENCES** — Notification Preferences, Theme (value shown, e.g. "System Default").
6. **SUPPORT** — Help & Support, Report a Problem, About HostelHub.
7. Logout (same red treatment as the drawer).
8. Version string at the bottom.

Same `InfoRow` component as Profile/My Hostel Details — do not build a third variant of this row pattern.

6.4 HOSTEL SERVICES

No reference screenshots exist for these six — built by extension from the established system. Each uses `AppBackHeader` (← + page title), the service's color pair from Section 3.1, and existing shared components. Flag to the project owner before treating any of these as final per Assumption 4.

Screen	Suggested content	Reused components
Room	Expanded Room Snapshot (block/room/floor), roommates list with contact actions, room-specific repair/maintenance history, link to hostel rules	<code>InfoRow</code> , roommate rows from Profile, <code>EmptyState</code>
Mess	Full weekly menu (Breakfast/Lunch/Snacks/Dinner per day), mess timings, notices/announcements, optional feedback action	<code>SectionHeader</code> , <code>AppCard</code>
Laundry	Slot booking flow, current booking status card, booking history	<code>ScheduleCard</code> -style status card, <code>PrimaryButton</code> , <code>EmptyState</code> ("No laundry bookings.")
Repair	"Raise a Request" primary action, active request status tracker (same Reported/Assigned/Scheduled/Completed pattern as Home), history list	Status tracker from Home's Active Repair, <code>EmptyState</code> ("No upcoming repairs.")
Events	Upcoming events list (<code>HighlightCard</code> -style), event detail with Register/RSVP action, past events optionally	<code>HighlightCard</code> , <code>PrimaryButton</code> , <code>EmptyState</code> ("No upcoming events.")
Resources	Hostel facilities/resources list (Wi-Fi info, library, gym, sports room, common room, warden contact, emergency contacts)	<code>ServiceCard</code> / <code>InfoRow</code> list

Do not add booking fees, deposits, or any payment step to Laundry, Repair, or Room.

6.5 SECONDARY

Also built by extension — same caveat as 6.4. All use `AppBackHeader`.

- **My Hostel Details** — hostel rules, timings, warden contact, block/room reference info. Reached from Profile's "View Hostel Details" row and the drawer's "My Hostel Details."
- **Settings** — notification preferences link, theme selector (System/Light/Dark, matching the "System Default" value seen in the Account Panel), language, change password, linked email/mobile.
- **Notification Preferences** — toggle list, one per alert category (Smart Alerts, Notices, Updates) and per service (Mess, Laundry, Repair, Events), using standard switch styling in `primary`.
- **Help & Support** — FAQ list (expandable rows) + a "Contact Support" card/action.
- **Report a Problem** — category selector, description text area, optional attachment, `PrimaryButton` "Submit."
- **About** — brand lockup, version string, Terms of Service / Privacy Policy links.

7. Mock Data Contract

Centralize all mock data in `mock_data/` — never duplicate the same values inline across screens, and never hard-code the specific example dates below as permanent (prefer "Today"/"Tomorrow"/relative phrasing; use a concrete date only where a UI genuinely needs one, sourced from the mock layer, not typed inline in a widget).

```
Student:
  name: "Sid", id: "STU1024", block: "B", room: "304", floor: "3rd Floor",
  roommates: 2, course: "Computer Science Engineering", batch: "2024 - 2028"

Laundry: "Tomorrow, 10:00 AM"
Repair:  "Fan not working", "Tomorrow, 2:00 PM - 4:00 PM", technician: "Arun"
Mess:    "Dinner menu updated"
Event:   "Hostel Cricket Cup", "20 Aug, 5:00 PM"
```

Model shapes needed at minimum: `Student`, `RoomInfo`, `Roommate`, `ScheduleItem`, `Alert`, `Notice`, `Event`, `RepairRequest`, `MessMenu`, `LaundryBooking`, `Resource`.

8. States: Loading / Empty / Error

- **Loading** — every data-driven section must support a loading state: skeletons, subtle progress indicators, placeholders. No large blocking full-screen spinners unless truly unavoidable.
- **Empty** — every list-based feature needs an empty state using the standard typography/card system (e.g. "No upcoming repairs.", "No alerts right now.", "No laundry bookings.", "No upcoming events.").
- **Error** — friendly, non-technical: "Something went wrong." / "We couldn't load your schedule." + `PrimaryButton` "Try Again", using existing components — no custom one-off error screens.

9. Motion & Accessibility

Animation — subtle only: page transitions, drawer/panel slide, card fade/slide, tab selection, button feedback, loading shimmer. No bouncing, particle effects, or gaming-style flourishes. Motion should aid usability, not decorate.

Accessibility — readable contrast, adequately large tap targets, semantic labels, screen-reader-friendly buttons, scalable text where technically feasible. Never communicate status through color alone — pair color with a label or icon (e.g. not just a green dot for "Completed," but "Completed ✓").

Responsive — must work across mobile screen sizes: flexible widths, safe areas, scrolling, adaptive layouts, sensible min/max widths. The reference screenshots are visual references, not fixed pixel canvases. Content must never sit underneath the bottom navigation.

10. Absolute Guardrails — Do Not

- Add payments, fees, billing, wallet functionality, transactions, or fee reminders — anywhere, ever.
 - Change the primary color or font family.
 - Create a second design system, random card styles, random button styles, gradients, heavy shadows, or excessive animation.
 - Mix unrelated icon libraries.
 - Redesign another team member's page, or any screen already matching spec.
 - Remove existing functionality or change navigation without approval.
 - Create duplicate shared components, or hard-code repeated design values instead of using the theme tokens.
 - Use fixed screen-sized layouts.
 - Add backend logic or external services unless the task explicitly requires it.
 - Add a new package/dependency unless it provides a clear, necessary benefit — check the project and existing dependencies first, and report every addition.
 - Redesign the Home page, restructure the project, replace the navigation system or theme, or refactor unrelated code while implementing an unrelated single feature.
-

11. Required Workflow For Every Task

```
READ DESIGN RULES → INSPECT PROJECT → INSPECT EXISTING COMPONENTS  
→ IDENTIFY REUSABLE COMPONENTS → PLAN FEATURE → IMPLEMENT  
→ TEST → CHECK DESIGN CONSISTENCY → REPORT CHANGES
```

Don't start creating files before inspecting the existing project. Before modifying a high-risk shared file (theme, navigation, app shell, shared widgets, global constants, routing, shared models/services): confirm the change is actually necessary, make the smallest reasonable edit, preserve backward compatibility, verify existing screens still work, and report it.

12. What To Report When You're Done

1. Files created
2. Files modified
3. Components reused
4. Components created
5. Navigation added/changed

6. Mock data added
7. Dependencies added (if any)
8. Any unresolved issues

Never claim something works unless you actually checked it.

13. Definition of Done

Visual — colors/typography/spacing match the system · cards use standard radius · buttons use standard styles · icons are consistent · navigation matches the app · the screen visibly belongs to HostelHub.

Functional — navigation works · buttons work · scrolling works · back navigation works · empty states exist where needed · loading states are considered · nothing is hidden behind the bottom nav.

Code — existing components were reused · no unnecessary duplicate components · no unnecessary dependencies · design values are centralized (theme tokens, not inline hex/px) · mock data is separated from UI · unrelated features weren't touched.

14. Final Design Principle

The goal is not "make every screen identical." The goal is "make every screen feel like it belongs to the same product." Different services can carry different content and semantic colors, but every screen shares: typography, spacing, cards, buttons, navigation, icon style, header style, interaction patterns, and design tokens.

15. Team Ownership (for context, not enforcement by the agent)

- **Member 1 — Core + Home:** Entry (Splash/Onboarding/Login), Core (theme, shared components, navigation), Home. Owns the global design system.
- **Member 2 — Hostel Services:** Room, Mess, Laundry, Repair, Events, Resources. Must consume the shared theme/cards/buttons/headers/typography/spacing/service colors.
- **Member 3 — Personal + Communication:** Schedule, Alerts, Profile, Hamburger Menu, Account Panel, Settings, Notification Preferences, Help & Support, Report a Problem, About.

If you don't know which member is asking, don't guess ownership — just follow the shared-file rules in Section 11.

16. Suggested Build Order (if running this in phases)

If Copilot Agent struggles to hold the entire scope in one pass, split it into sequential prompts, each referencing this file:

1. **Foundation** — theme tokens (Section 3), navigation shell (Section 5), and the core component library (Section 4).
2. **Entry** — Splash, Onboarding, Login (6.1).
3. **Main App** — Home, Schedule, Alerts, Profile (6.2).
4. **Overlays** — Hamburger Drawer, Account Panel (6.3).
5. **Hostel Services** — Room, Mess, Laundry, Repair, Events, Resources (6.4).
6. **Secondary** — My Hostel Details, Settings, Notification Preferences, Help & Support, Report a Problem, About (6.5).

After each phase, run the Definition of Done checklist (Section 13) before moving to the next.